How r(txt[j ...j+m−1]) was updated from previous r(txt[j−1...j+ m−2])?

Upon experimenting with the examples given in the provided materials, I was able to determine a pattern in how the value changes from one iteration to the next. Since we multiply different numbers by powers of beta from range 0 to some value p, which is the length of the pattern we are checking for, we can reuse the previous iteration's value to calculate the next. That means that each check is O(1).

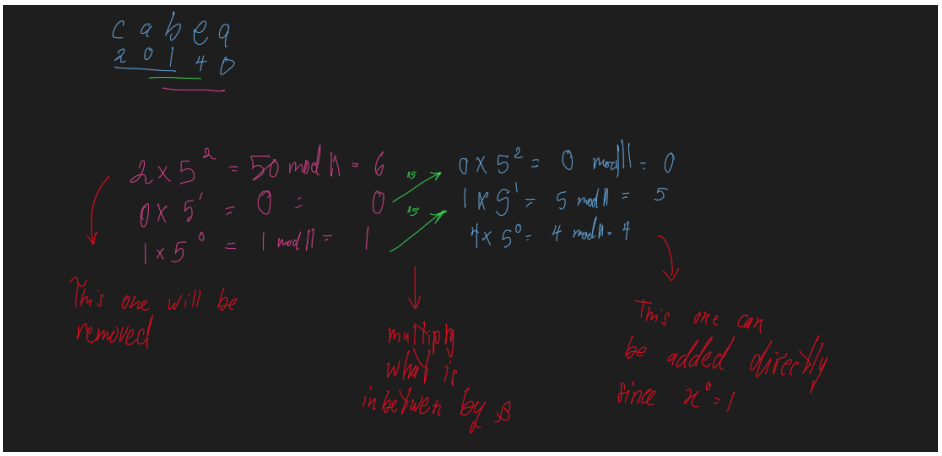
Since we have a sliding window that removes one character from the window's left side and adds one from the right side, we can replicate this by doing the following:

1- Subtract the value of the leftmost character's hash. This is done because the new window does not include it.

2- Multiply the remaining value by beta, which replicates the shifting of each character by using mathematics to skip extra computation. Since positions at the left hold higher powers and the difference between each character is a single power of beta then this is doable.

3- Add the newly included character in the window to the value directly since it is being multiplied by beta^0 = 1.

Below we can see a diagram outlining this operation:



At each calculation we make sure to modulo by the prime number used in execution to make sure that the values do not grow too large and keep them computationally manageable. Python's modulo operation also is useful since it does not return negative values which makes implementation easier. Upon a hash match we must explicitly compare characters in order to confirm a match's presence. It is also important for our modulo to be a prime number to reduce the probability of hash collisions.